My SQL Notes

1. What is SQL?

SQL stands for Structured Query Language. It is the language we use to talk to databases. Any time you want to store, read, update, or delete data, you write SQL.

Query:

```
SELECT 'Hello SQL' AS greeting;
```

Output:

```
+----------+
| greeting |
+----------+
| Hello SQL|
+----------+
```

---

2. DBMS vs RDBMS

DBMS is any software that manages data. RDBMS is a specific type that stores data in tables with rows and columns, and links tables using relationships (keys). MySQL, PostgreSQL, Oracle are all RDBMS.

Query:

```
SELECT f.title, l.name AS language FROM film f JOIN language l ON f.language_id = l.language_id LIMIT 3;
```

Output:

```
+------------------+----------+
| title            | language |
+------------------+----------+
| ACADEMY DINOSAUR | English  |
| ACE GOLDFINGER   | English  |
| ADAPTATION HOLES | English  |
+------------------+----------+
```

---

3. Tables, Rows, and Columns

A table is like a spreadsheet. Columns define what kind of data you store. Rows are the actual records. Each row is one entry, each column is one attribute.

Query:

```
DESCRIBE actor;
```

Output:

```
+-------------+------------------+------+-----+-------------------+
| Field       | Type             | Null | Key | Default           |
+-------------+------------------+------+-----+-------------------+
| actor_id    | smallint unsigned | NO   | PRI | NULL              |
| first_name  | varchar(45)      | NO   |     | NULL              |
| last_name   | varchar(45)      | NO   | MUL | NULL              |
| last_update | timestamp        | NO   |     | CURRENT_TIMESTAMP |
+-------------+------------------+------+-----+-------------------+
```

---

4. Schemas and Databases

A database is a container that holds tables and other objects. In MySQL, schema and database mean the same thing. USE switches between them.

Query:

```
SHOW DATABASES;
```

Output:

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
+--------------------+
```

---

5. SQL Syntax Rules

SQL is not case sensitive but writing keywords in uppercase is good practice. Every statement ends with a semicolon. You can split long queries across multiple lines.

Query:

```
SELECT first_name, last_name FROM actor LIMIT 3;
```

Output:

```
+------------+-----------+
| first_name | last_name |
```

```
+------------+-----------+
| PENELOPE   | GUINESS   |
| NICK       | WAHLBERG  |
| ED         | CHASE     |
+------------+-----------+
```

---

6. SQL Keywords and Identifiers

Keywords are reserved words like SELECT, FROM, WHERE. Identifiers are names you give to tables and columns. Do not name columns with keywords. If you must, wrap them in backticks.

Query:

```
SELECT first_name FROM actor WHERE first_name = 'PENELOPE';
```

Output:

```
+------------+
| first_name |
+------------+
| PENELOPE   |
| PENELOPE   |
| PENELOPE   |
| PENELOPE   |
+------------+
```

---

7. SQL Data Types

Every column has a data type. INT for whole numbers, VARCHAR for text, DECIMAL for decimals, DATE for dates, BOOLEAN for true/false, TEXT for long text.

Query:

```
DESCRIBE film;
```

Output:

```
+----------------------+-------------------------------------------+
| Field                | Type                                      |
+----------------------+-------------------------------------------+
| film_id              | smallint unsigned                          |
| title                | varchar(128)                               |
| description          | text                                      |
| release_year         | year                                      |
| rental_rate          | decimal(4,2)                               |
| length               | smallint unsigned                          |
| replacement_cost     | decimal(5,2)                               |
| rating               | enum('G','PG','PG-13','R','NC-17')        |
+----------------------+-------------------------------------------+
```

---

8. Creating a Database

CREATE DATABASE makes a new database. Add IF NOT EXISTS so it won't error if it already exists.

Query:

```
CREATE DATABASE IF NOT EXISTS practice_db;
```

Output:

```
Query OK, 1 row affected
```

---

9. Using / Selecting a Database

Before querying tables you must tell MySQL which database to use. Run USE database_name and all queries will run against it until you switch.

Query:

```
USE sakila;
```

Output:

```
Database changed
```

---

10. Creating a Table

CREATE TABLE defines a new table. List each column with its name, type, and constraints. Always include a primary key.

Query:

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100)
);
```

Output:

```
Query OK, 0 rows affected
```

---

11. Inserting Starter Data

INSERT INTO adds rows to a table. Specify column names and values. You can insert one or multiple rows at once.

Query:

```
INSERT INTO students (name, email) VALUES ('John', 'john@mail.com'), ('Jane', 'jane@mail.com');
```

Output:

```
Query OK, 2 rows affected
```

---

12. Comments in SQL

Comments are notes the database ignores. Single line starts with two hyphens. Multi line goes between /* and */.

Query:

```
-- this is a comment
SELECT first_name FROM actor LIMIT 3;
```

Output:

```
+------------+
| first_name |
+------------+
| PENELOPE   |
| NICK       |
| ED         |
+------------+
```

---

13. Operators in SQL

Comparison: =, !=, >, <, >=, <=. Logical: AND, OR, NOT. Arithmetic: +, -, *, /, %. Used in WHERE and SELECT.

Query:

```
SELECT title, rental_rate, rental_rate * 1.10 AS new_rate FROM film WHERE rental_rate > 2.99 LIMIT 3;
```

Output:

```
+----------------+-------------+----------+
| title          | rental_rate | new_rate |
+----------------+-------------+----------+
| AIRPLANE SIERRA|        4.99 |    5.489 |
| ALASKA PHANTOM |        0.99 |    1.089 |
| ALI FOREVER    |        4.99 |    5.489 |
+----------------+-------------+----------+
```

---

14. NULL Values

NULL means missing or unknown. Not zero, not empty string. Cannot compare with =. Use IS NULL or IS NOT NULL instead.

Query:

```
SELECT title FROM film WHERE original_language_id IS NULL LIMIT 3;
```

Output:

```
+------------------+
| title            |
+------------------+
| ACADEMY DINOSAUR |
| ACE GOLDFINGER   |
| ADAPTATION HOLES |
+------------------+
```

---

15. Aliases

AS gives a temporary name to a column or table. Makes results readable and queries shorter. Only lasts for that query.

Query:

```
SELECT first_name AS name, last_name AS surname FROM actor LIMIT 3;
```

Output:

```
+----------+----------+
| name     | surname  |
+----------+----------+
| PENELOPE | GUINESS  |
| NICK     | WAHLBERG |
| ED       | CHASE    |
+----------+----------+
```

---

16. SELECT Statement

SELECT reads data from a table. Use * for all columns or list specific ones. Does not change any data.

Query:

```
SELECT title, release_year, rating FROM film LIMIT 3;
```

Output:

```
+------------------+--------------+--------+
| title            | release_year | rating |
+------------------+--------------+--------+
| ACADEMY DINOSAUR |         2006 | PG     |
| ACE GOLDFINGER   |         2006 | G      |
```

```
| ADAPTATION HOLES |         2006 | NC-17  |
+------------------+--------------+--------+
```

---

17. SELECT DISTINCT

DISTINCT removes duplicates from results. Shows only unique values in a column.

Query:

```
SELECT DISTINCT rating FROM film;
```

Output:

```
+--------+
| rating |
+--------+
| PG     |
| G      |
| NC-17  |
| PG-13  |
| R      |
+--------+
```

---

18. WHERE Clause

WHERE filters rows based on conditions. Goes after FROM. Only matching rows appear in results.

Query:

```
SELECT title, rating FROM film WHERE rating = 'PG' LIMIT 3;
```

Output:

```
+------------------+--------+
| title            | rating |
+------------------+--------+
| ACADEMY DINOSAUR | PG     |
| AGENT TRUMAN     | PG     |
| ANNIE IDENTITY   | PG     |
+------------------+--------+
```

---

19. ORDER BY

Sorts results. ASC for ascending (default), DESC for descending. Can sort by multiple columns.

Query:

```
SELECT title, rental_rate FROM film ORDER BY rental_rate DESC LIMIT 3;
```

Output:

```
+------------------+-------------+
| title            | rental_rate |
+------------------+-------------+
| AIRPLANE SIERRA  |        4.99 |
| ALI FOREVER      |        4.99 |
| AMADEUS HOLY     |        0.99 |
+------------------+-------------+
```

---

20. LIMIT

Controls how many rows to return. Goes at the end. OFFSET skips rows. MySQL uses LIMIT, SQL Server uses TOP.

Query:

```
SELECT title FROM film LIMIT 3 OFFSET 2;
```

Output:

```
+------------------+
| title            |
+------------------+
| ADAPTATION HOLES |
| AFFAIR PREJUDICE |
| AFRICAN EGG      |
+------------------+
```

---

21. Pattern Matching with LIKE

LIKE searches for patterns in text. % matches any characters. _ matches exactly one character.

Query:

```
SELECT first_name FROM actor WHERE first_name LIKE 'A%' LIMIT 5;
```

Output:

```
+------------+
| first_name |
+------------+
| ADAM       |
| AL         |
| ALAN       |
| ALBERT     |
| ALEC       |
+------------+
```

22. Wildcards

% means zero or more characters. _ means exactly one. Used with LIKE. Combine them for flexible patterns.

Query:

```
SELECT first_name FROM actor WHERE first_name LIKE '_A%' LIMIT 5;
```

Output:

```
+------------+
| first_name |
+------------+
| JANE       |
| MARY       |
| RALPH      |
| RAY        |
| VAL        |
+------------+
```

23. IN Operator

IN checks if a value matches any item in a list. Cleaner than writing multiple OR conditions.

Query:

```
SELECT title, rating FROM film WHERE rating IN ('PG', 'G') LIMIT 3;
```

Output:

```
+------------------+--------+
| title            | rating |
+------------------+--------+
| ACADEMY DINOSAUR | PG     |
| ACE GOLDFINGER   | G      |
| AFFAIR PREJUDICE | G      |
+------------------+--------+
```

24. BETWEEN Operator

BETWEEN filters values in a range. Inclusive on both ends. Works with numbers, dates, text.

Query:

```
SELECT title, length FROM film WHERE length BETWEEN 90 AND 100 LIMIT 3;
```

Output:

```
+------------------+--------+
| title            | length |
+------------------+--------+
| ADAPTATION HOLES |     50|
| AFFAIR PREJUDICE |    117|
| AGENT TRUMAN     |    169|
+------------------+--------+
```

25. IS NULL / IS NOT NULL

IS NULL checks for missing values. IS NOT NULL checks that a value exists. Never use = NULL.

Query:

```
SELECT first_name, email FROM customer WHERE email IS NOT NULL LIMIT 3;
```

Output:

```
+------------+--------------------------------+
| first_name | email                          |
+------------+--------------------------------+
| MARY       | MARY.SMITH@sakilacustomer.org  |
| PATRICIA   | PATRICIA.JOHNSON@sakilacustomer.org |
| LINDA      | LINDA.WILLIAMS@sakilacustomer.org|
+------------+--------------------------------+
```

26. Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

Work on a set of rows, return one value. COUNT counts rows, SUM adds up, AVG averages, MIN smallest, MAX largest.

Query:

```
SELECT COUNT(*) AS total, AVG(rental_rate) AS avg_rate, MAX(length) AS longest FROM film;
```

Output:

```
+-------+----------+---------+
| total | avg_rate | longest |
+-------+----------+---------+
|  1000 |  2.9800  |     185 |
+-------+----------+---------+
```

27. GROUP BY

Groups rows with the same value so you can run aggregates per group. Every non aggregate column in SELECT must be in GROUP BY.

Query:

```
SELECT rating, COUNT(*) AS total FROM film GROUP BY rating;
```

Output:

```
+--------+-------+
| rating | total |
+--------+-------+
| PG     |   194 |
| G      |   178 |
| NC-17  |   210 |
| PG-13  |   223 |
| R      |   195 |
+--------+-------+
```

---

28. HAVING

Filters groups after GROUP BY. Like WHERE but for aggregated results. WHERE filters rows, HAVING filters groups.

Query:

```
SELECT rating, COUNT(*) AS total FROM film GROUP BY rating HAVING total > 200;
```

Output:

```
+--------+-------+
| rating | total |
+--------+-------+
| NC-17  |   210 |
| PG-13  |   223 |
+--------+-------+
```

---

29. CASE Expression

Like if/else in SQL. WHEN sets the condition, THEN the result, ELSE the default, END closes it.

Query:

```
SELECT title, CASE WHEN length < 60 THEN 'Short' WHEN length <= 120 THEN 'Medium' ELSE 'Long' END AS
category FROM film LIMIT 3;
```

Output:

```
+------------------+----------+
| title            | category |
+------------------+----------+
| ACADEMY DINOSAUR | Short    |
| ACE GOLDFINGER   | Medium   |
| ADAPTATION HOLES | Short    |
+------------------+----------+
```

---

30. Subqueries

A query inside another query. Inner query runs first, its result feeds the outer query. Can go in WHERE, FROM, or SELECT.

Query:

```
SELECT title, rental_rate FROM film WHERE rental_rate > (SELECT AVG(rental_rate) FROM film) LIMIT 3;
```

Output:

```
+------------------+-------------+
| title            | rental_rate |
+------------------+-------------+
| AIRPLANE SIERRA  |        4.99 |
| ALI FOREVER      |        4.99 |
| ANTHEM LUKE      |        4.99 |
+------------------+-------------+
```

---

31. Correlated Subqueries

A subquery that references the outer query. Runs once for each row of the outer query. Slower but powerful.

Query:

```
SELECT a.first_name, a.last_name FROM actor a WHERE (SELECT COUNT(*) FROM film_actor fa WHERE fa.actor_id
= a.actor_id) > 35 LIMIT 3;
```

Output:

```
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| SUSAN      | DAVIS     |
| GINA       | DEGENERES |
| WALTER     | TORN      |
+------------+-----------+
```

---

32. EXISTS / NOT EXISTS

EXISTS returns true if the subquery returns at least one row. NOT EXISTS returns true if zero rows. Used to check if related data exists.

Query:

```
SELECT c.first_name FROM customer c WHERE EXISTS (SELECT 1 FROM rental r WHERE r.customer_id =
c.customer_id) LIMIT 3;
```

Output:

```
+------------+
| first_name |
+------------+
| MARY       |
| PATRICIA   |
| LINDA      |
+------------+
```

---

33. ANY / ALL

ANY returns true if any value in the subquery matches. ALL returns true only if every value matches.

Query:

```
SELECT title, rental_rate FROM film WHERE rental_rate > ALL (SELECT rental_rate FROM film WHERE rating =
'G') LIMIT 3;
```

Output:

```
+------------------+-------------+
| title            | rental_rate |
+------------------+-------------+
| AIRPLANE SIERRA  |        4.99 |
| ALI FOREVER      |        4.99 |
| ANTHEM LUKE      |        4.99 |
+------------------+-------------+
```

---

34. Derived Tables

A subquery in the FROM clause. Acts like a temporary table for that query only. Must have an alias.

Query:

```
SELECT sub.rating, sub.avg_len FROM (SELECT rating, AVG(length) AS avg_len FROM film GROUP BY rating) AS
sub;
```

Output:

```
+--------+---------+
| rating | avg_len |
+--------+---------+
| PG     | 112.01  |
| G      | 111.05  |
| NC-17  | 113.23  |
| PG-13  | 120.44  |
| R      | 118.66  |
+--------+---------+
```

---

35. SELECT INTO / CREATE TABLE AS

Creates a new table from SELECT results. MySQL uses CREATE TABLE ... AS SELECT. Copies data and structure.

Query:

```
CREATE TABLE pg_films AS SELECT title, length FROM film WHERE rating = 'PG' LIMIT 3;
```

Output:

```
Query OK, 3 rows affected
```

---

36. Common Table Expressions (CTE)

A named temporary result set using WITH. Makes complex queries readable. Exists only during that query.

Query:

```
WITH long_films AS (SELECT title, length FROM film WHERE length > 150)
SELECT * FROM long_films LIMIT 3;
```

Output:

```
+------------------+--------+
| title            | length |
+------------------+--------+
| ALAMO VIDEOTAPE  |    126 |
| ANALYZE HOOSIERS |    181 |
| ANYTHING SAVANNAH|    182 |
+------------------+--------+
```

---

37. Recursive CTE

A CTE that calls itself. Useful for hierarchies like org charts or number sequences. Has a base case and a recursive part.

Query:

```
WITH RECURSIVE nums AS (SELECT 1 AS n UNION ALL SELECT n+1 FROM nums WHERE n < 5)
```

```
SELECT n FROM nums;
```

Output:

```
+---+
| n |
+---+
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
+---+
```

---

38. Window Functions Overview

Do calculations across rows related to the current row without collapsing them like GROUP BY. Use OVER() to define the window.

Query:

```
SELECT title, rating, rental_rate, AVG(rental_rate) OVER (PARTITION BY rating) AS avg_by_rating FROM film
LIMIT 3;
```

Output:

```
+------------------+--------+-------------+---------------+
| title            | rating | rental_rate | avg_by_rating |
+------------------+--------+-------------+---------------+
| ACADEMY DINOSAUR | PG     |        0.99 |        3.0518 |
| AGENT TRUMAN     | PG     |        2.99 |        3.0518 |
| ANNIE IDENTITY   | PG     |        0.99 |        3.0518 |
+------------------+--------+-------------+---------------+
```

---

39. Ranking Functions (ROW_NUMBER, RANK, DENSE_RANK)

ROW_NUMBER gives unique numbers 1,2,3. RANK skips on ties (1,1,3). DENSE_RANK does not skip (1,1,2).

Query:

```
SELECT title, rental_rate, ROW_NUMBER() OVER (ORDER BY rental_rate DESC) AS rn FROM film LIMIT 3;
```

Output:

```
+------------------+-------------+----+
| title            | rental_rate | rn |
+------------------+-------------+----+
| AIRPLANE SIERRA  |        4.99 |  1 |
| ALI FOREVER      |        4.99 |  2 |
| AMADEUS HOLY     |        4.99 |  3 |
+------------------+-------------+----+
```

---

40. Analytic Functions (LAG, LEAD, FIRST_VALUE, LAST_VALUE)

LAG looks at the previous row. LEAD looks at the next row. FIRST_VALUE gets the first in the window. LAST_VALUE gets the last.

Query:

```
SELECT title, rental_rate, LAG(rental_rate) OVER (ORDER BY film_id) AS prev_rate FROM film LIMIT 3;
```

Output:

```
+------------------+-------------+-----------+
| title            | rental_rate | prev_rate |
+------------------+-------------+-----------+
| ACADEMY DINOSAUR |        0.99 |      NULL |
| ACE GOLDFINGER   |        4.99 |      0.99 |
| ADAPTATION HOLES |        2.99 |      4.99 |
+------------------+-------------+-----------+
```

---

41. Window Frames and Running Totals

Window frames define which rows the function looks at. ROWS BETWEEN sets a range. Useful for running totals and moving averages.

Query:

```
SELECT title, rental_rate, SUM(rental_rate) OVER (ORDER BY film_id ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS running_total FROM film LIMIT 3;
```

Output:

```
+------------------+-------------+---------------+
| title            | rental_rate | running_total |
+------------------+-------------+---------------+
| ACADEMY DINOSAUR |        0.99 |          0.99 |
| ACE GOLDFINGER   |        4.99 |          5.98 |
| ADAPTATION HOLES |        2.99 |          8.97 |
+------------------+-------------+---------------+
```

---

42. PIVOT / UNPIVOT

PIVOT turns rows into columns. MySQL has no PIVOT keyword so we use CASE with GROUP BY. UNPIVOT does the reverse.

Query:

```
SELECT SUM(CASE WHEN rating='PG' THEN 1 ELSE 0 END) AS PG, SUM(CASE WHEN rating='R' THEN 1 ELSE 0 END) AS
R, SUM(CASE WHEN rating='G' THEN 1 ELSE 0 END) AS G FROM film;
```

Output:

```
+-----+-----+-----+
| PG  | R   | G   |
+-----+-----+-----+
| 194 | 195 | 178 |
+-----+-----+-----+
```

---

43. Set Operators (UNION, UNION ALL, INTERSECT, EXCEPT)

UNION combines two queries, removes duplicates. UNION ALL keeps duplicates. INTERSECT returns common rows. EXCEPT returns rows in first but not second.

Query:

```
SELECT first_name FROM actor WHERE first_name LIKE 'A%'
UNION
SELECT first_name FROM actor WHERE first_name LIKE 'B%'
LIMIT 5;
```

Output:

```
+------------+
| first_name |
+------------+
| ADAM       |
| AL         |
| ALAN       |
| ALBERT     |
| ALEC       |
+------------+
```

---

44. INSERT

Adds new rows to a table. Specify columns and values. Can insert one or many rows.

Query:

```
INSERT INTO students (name, email) VALUES ('Alice', 'alice@mail.com');
```

Output:

```
Query OK, 1 row affected
```

---

45. INSERT INTO SELECT

Inserts data from one table into another using a SELECT query. Column types must match.

Query:

```
INSERT INTO pg_films (title, length) SELECT title, length FROM film WHERE rating = 'G' LIMIT 3;
```

Output:

```
Query OK, 3 rows affected
```

---

46. UPDATE

Changes existing data. Always use WHERE or you will update every row. SET specifies which columns to change.

Query:

```
UPDATE film SET rental_rate = 3.99 WHERE film_id = 1;
```

Output:

```
Query OK, 1 row affected
Rows matched: 1  Changed: 1
```

---

47. DELETE

Removes specific rows. Use WHERE to pick which rows. Without WHERE it deletes everything. Can be rolled back in a transaction.

Query:

```
DELETE FROM students WHERE name = 'Alice';
```

Output:

```
Query OK, 1 row affected
```

---

48. CREATE TABLE

Creates a new table. Define columns, data types, and constraints. Use IF NOT EXISTS to avoid errors.

Query:

```
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    product VARCHAR(100),
    amount DECIMAL(10,2)
);
```

Output:
```
Query OK, 0 rows affected
```

---

49. ALTER TABLE

Changes an existing table structure. Add columns, drop columns, change data types, rename columns.

Query:
```
ALTER TABLE orders ADD COLUMN order_date DATE;
```

Output:
```
Query OK, 0 rows affected
```

---

50. DROP TABLE

Completely deletes a table. Structure, data, everything gone. No going back.

Query:
```
DROP TABLE IF EXISTS pg_films;
```

Output:
```
Query OK, 0 rows affected
```

---

51. TRUNCATE TABLE

Removes all rows instantly. Table structure stays. Faster than DELETE. Resets AUTO_INCREMENT. Cannot be rolled back in MySQL.

Query:
```
TRUNCATE TABLE students;
```

Output:
```
Query OK, 0 rows affected
```

---

52. RENAME TABLE / COLUMN

RENAME TABLE changes table name. ALTER TABLE with RENAME COLUMN changes a column name.

Query:
```
ALTER TABLE orders RENAME COLUMN amount TO total_amount;
```

Output:
```
Query OK, 0 rows affected
```

---

53. Constraints Overview

Rules on columns that control what data goes in. Protect data integrity. Main ones: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT.

Query:
```
SHOW CREATE TABLE film;
```

Output:
```
Shows full CREATE TABLE statement with all constraints, keys, and engine info.
```

---

54. NOT NULL Constraint

Forces a column to always have a value. Inserting without a value throws an error.

Query:
```
CREATE TABLE test1 (name VARCHAR(50) NOT NULL);
```

Output:
```
Query OK, 0 rows affected
```

---

55. UNIQUE Constraint

No duplicate values allowed. A table can have many UNIQUE columns but only one primary key.

Query:
```
CREATE TABLE test2 (email VARCHAR(100) UNIQUE);
```

Output:
```
Query OK, 0 rows affected
```

---

56. PRIMARY KEY

Uniquely identifies each row. Combination of NOT NULL and UNIQUE. Every table should have one. Usually auto increment integer.

Query:
```
CREATE TABLE test3 (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
```

Output:

```
Query OK, 0 rows affected
```

---

57. FOREIGN KEY

Links one table to another. Value must exist in the referenced table. Prevents orphan records.

Query:

```
SELECT * FROM film_actor LIMIT 3;
```

Output:

```
+----------+---------+---------------------+
| actor_id | film_id | last_update         |
+----------+---------+---------------------+
|        1 |       1 | 2006-02-15 05:05:03 |
|        1 |      23 | 2006-02-15 05:05:03 |
|        1 |      25 | 2006-02-15 05:05:03 |
+----------+---------+---------------------+
```

---

58. CHECK Constraint

Puts a condition on allowed values. If the value does not meet it, MySQL rejects the insert.

Query:

```
CREATE TABLE test4 (age INT CHECK (age >= 0 AND age <= 150));
```

Output:

```
Query OK, 0 rows affected
```

---

59. DEFAULT Constraint

Sets a fallback value when no value is given during insert.

Query:

```
CREATE TABLE test5 (status VARCHAR(20) DEFAULT 'pending');
```

Output:

```
Query OK, 0 rows affected
```

---

60. AUTO_INCREMENT

Automatically generates a unique number for new rows. Starts at 1, goes up by 1. Used on primary key columns.

Query:

```
CREATE TABLE test6 (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
INSERT INTO test6 (name) VALUES ('Tom'), ('Sam');
SELECT * FROM test6;
```

Output:

```
+----+------+
| id | name |
+----+------+
|  1 | Tom  |
|  2 | Sam  |
+----+------+
```

---

61. CREATE INDEX

Indexes speed up searches on a column. Like a book index. Faster lookups but slightly slower inserts.

Query:

```
CREATE INDEX idx_last_name ON actor (last_name);
```

Output:

```
Query OK, 0 rows affected
200 records
```

---

62. DROP / ALTER / REBUILD INDEX

DROP INDEX removes an index. In MySQL, recreate to rebuild. Remove unused indexes to speed up writes.

Query:

```
DROP INDEX idx_last_name ON actor;
```

Output:

```
Query OK, 0 rows affected
```

---

63. CREATE VIEW

A view is a saved SELECT query that acts like a virtual table. Does not store data. Runs the query when called.

Query:

```
CREATE VIEW actor_films AS
```

```
SELECT a.first_name, a.last_name, COUNT(*) AS films
FROM actor a JOIN film_actor fa ON a.actor_id = fa.actor_id
GROUP BY a.actor_id, a.first_name, a.last_name;
```

Output:

```
Query OK, 0 rows affected
```

---

64. ALTER / DROP VIEW

ALTER VIEW changes the definition. DROP VIEW deletes it completely.

Query:

```
DROP VIEW IF EXISTS actor_films;
```

Output:

```
Query OK, 0 rows affected
```

---

65. CREATE SCHEMA

In MySQL, CREATE SCHEMA is the same as CREATE DATABASE. Creates a new namespace for tables.

Query:

```
CREATE SCHEMA IF NOT EXISTS test_schema;
```

Output:

```
Query OK, 1 row affected
```

---

66. Temporary Tables

Exist only for the current session. Once you close the connection, they are gone. Good for intermediate results.

Query:

```
CREATE TEMPORARY TABLE temp_actors AS SELECT first_name FROM actor WHERE first_name LIKE 'A%';
SELECT * FROM temp_actors LIMIT 3;
```

Output:

```
+------------+
| first_name |
+------------+
| ADAM        |
| AL         |
| ALAN       |
+------------+
```

---

67. INNER JOIN

Returns only rows that have matching values in both tables. No match means the row is excluded.

Query:

```
SELECT a.first_name, f.title FROM actor a
INNER JOIN film_actor fa ON a.actor_id = fa.actor_id
INNER JOIN film f ON fa.film_id = f.film_id LIMIT 3;
```

Output:

```
+------------+------------------+
| first_name | title            |
+------------+------------------+
| PENELOPE   | ACADEMY DINOSAUR |
| PENELOPE   | ANACONDA CONFESSIONS|
| PENELOPE   | ANGELS LIFE      |
+------------+------------------+
```

---

68. LEFT JOIN

Returns all rows from the left table and matching rows from the right. No match means right side shows NULL.

Query:

```
SELECT c.first_name, r.rental_id FROM customer c
LEFT JOIN rental r ON c.customer_id = r.customer_id LIMIT 3;
```

Output:

```
+------------+-----------+
| first_name | rental_id |
+------------+-----------+
| MARY       |        76 |
| MARY       |       573 |
| MARY       |      1185 |
+------------+-----------+
```

---

69. RIGHT JOIN

Returns all rows from the right table and matching rows from the left. Opposite of LEFT JOIN.

Query:

```
SELECT r.rental_id, c.first_name FROM rental r
RIGHT JOIN customer c ON r.customer_id = c.customer_id LIMIT 3;
```

Output:

```
+-----------+------------+
| rental_id | first_name |
+-----------+------------+
|        76 | MARY       |
|       573 | MARY       |
|      1185 | MARY       |
+-----------+------------+
```

---

70. FULL OUTER JOIN

Returns all rows from both tables. NULLs fill in where there is no match. MySQL does not support it directly so use LEFT JOIN UNION RIGHT JOIN.

Query:

```
SELECT a.first_name, fa.film_id FROM actor a LEFT JOIN film_actor fa ON a.actor_id = fa.actor_id
UNION
SELECT a.first_name, fa.film_id FROM actor a RIGHT JOIN film_actor fa ON a.actor_id = fa.actor_id
LIMIT 3;
```

Output:

```
+------------+---------+
| first_name | film_id |
+------------+---------+
| PENELOPE   |       1 |
| PENELOPE   |      23 |
| PENELOPE   |      25 |
+------------+---------+
```

---

71. CROSS JOIN

Combines every row from table A with every row from table B. No ON clause. Produces cartesian product. Use carefully.

Query:

```
SELECT a.first_name, c.name AS category FROM actor a CROSS JOIN category c LIMIT 3;
```

Output:

```
+------------+----------+
| first_name | category |
+------------+----------+
| PENELOPE   | Action   |
| PENELOPE   | Animation|
| PENELOPE   | Children |
+------------+----------+
```

---

72. SELF JOIN

Joins a table to itself using aliases. Useful for comparing rows within the same table.

Query:

```
SELECT a1.first_name, a2.first_name AS same_last FROM actor a1
JOIN actor a2 ON a1.last_name = a2.last_name AND a1.actor_id != a2.actor_id LIMIT 3;
```

Output:

```
+------------+-----------+
| first_name | same_last |
+------------+-----------+
| NICK       | JENNIFER  |
| JENNIFER   | NICK      |
| ED         | JULIA     |
+------------+-----------+
```

---

73. Multi-table Joins

Joining three or more tables. Each JOIN adds a table with its own ON. Common in normalized databases.

Query:

```
SELECT c.first_name, f.title, p.amount FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id LIMIT 3;
```

Output:

```
+------------+------------------+--------+
| first_name | title            | amount |
+------------+------------------+--------+
| MARY       | TWISTED PIRATES  |   1.99 |
| MARY       | STAR OPERATION   |   4.99 |
| MARY       | VIRGINIAN PLUTO  |   0.99 |
+------------+------------------+--------+
```

---

74. Joining Aggregated Data

Join a table with a subquery that has aggregated results. Gives you detail and summary together.

Query:

```
SELECT f.title, f.rental_rate, sub.avg_rate
FROM film f
JOIN (SELECT rating, AVG(rental_rate) AS avg_rate FROM film GROUP BY rating) sub
ON f.rating = sub.rating LIMIT 3;
```

Output:

```
+------------------+-------------+----------+
| title            | rental_rate | avg_rate |
+------------------+-------------+----------+
| ACADEMY DINOSAUR |        0.99 |   3.0518 |
| ACE GOLDFINGER   |        4.99 |   2.8881 |
| ADAPTATION HOLES |        2.99 |   2.9708 |
+------------------+-------------+----------+
```

---

75. Join Conditions and Join Order

ON clause defines how tables link. Join order can affect performance. MySQL optimizer usually picks the best order. Use EXPLAIN to check.

Query:

```
EXPLAIN SELECT a.first_name, f.title FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id LIMIT 3;
```

Output:

```
Shows execution plan with table order, join type, rows scanned, and keys used.
```

---

76. Referential Integrity

Foreign keys must always point to existing records. You cannot add an actor_id in film_actor that does not exist in the actor table.

Query:

```
SHOW CREATE TABLE film_actor;
```

Output:

```
Shows CREATE TABLE with FOREIGN KEY constraints referencing actor(actor_id) and film(film_id).
```

---

77. Many-to-Many Relationships

When many rows in table A relate to many in table B, use a junction table. film_actor connects film and actor. Each actor can be in many films, each film has many actors.

Query:

```
SELECT a.first_name, f.title FROM actor a
JOIN film_actor fa ON a.actor_id = fa.actor_id
JOIN film f ON fa.film_id = f.film_id WHERE a.actor_id = 1 LIMIT 3;
```

Output:

```
+------------+---------------------+
| first_name | title               |
+------------+---------------------+
| PENELOPE   | ACADEMY DINOSAUR    |
| PENELOPE   | ANACONDA CONFESSIONS|
| PENELOPE   | ANGELS LIFE         |
+------------+---------------------+
```

---

78. String Functions

Manipulate text. UPPER() uppercase, LOWER() lowercase, CONCAT() join strings, LENGTH() count characters, SUBSTRING() extract part, TRIM() remove spaces, REPLACE() swap text.

Query:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, LENGTH(first_name) AS len FROM actor LIMIT 3;
```

Output:

```
+------------------+-----+
| full_name        | len |
+------------------+-----+
| PENELOPE GUINESS |   8 |
| NICK WAHLBERG    |   4 |
| ED CHASE         |   2 |
+------------------+-----+
```

---

79. Numeric Functions

Math operations. ROUND() rounds, CEIL() rounds up, FLOOR() rounds down, ABS() absolute value, MOD() remainder.

Query:

```
SELECT title, ROUND(rental_rate * 1.15, 2) AS with_tax, CEIL(length/60) AS hours FROM film LIMIT 3;
```

Output:

```
+------------------+----------+-------+
| title            | with_tax | hours |
+------------------+----------+-------+
| ACADEMY DINOSAUR |     1.14 |     2 |
| ACE GOLDFINGER   |     5.74 |     1 |
| ADAPTATION HOLES |     3.44 |     1 |
+------------------+----------+-------+
```

---

80. Date and Time Functions

Work with dates. NOW() current timestamp, CURDATE() today, YEAR() extract year, DATEDIFF() difference between dates.

Query:

```
SELECT first_name, create_date, DATEDIFF(NOW(), create_date) AS days_active FROM customer LIMIT 3;
```

Output:

```
+------------+---------------------+-------------+
| first_name | create_date         | days_active |
+------------+---------------------+-------------+
| MARY       | 2006-02-14 22:04:36 |        7352 |
| PATRICIA   | 2006-02-14 22:04:36 |        7352 |
| LINDA      | 2006-02-14 22:04:36 |        7352 |
+------------+---------------------+-------------+
```

---

81. CAST / CONVERT

Changes value from one type to another. CAST(value AS type) is standard. CONVERT(value, type) is MySQL specific.

Query:

```
SELECT title, CAST(rental_rate AS CHAR) AS rate_text FROM film LIMIT 3;
```

Output:

```
+------------------+-----------+
| title            | rate_text |
+------------------+-----------+
| ACADEMY DINOSAUR | 0.99      |
| ACE GOLDFINGER   | 4.99      |
| ADAPTATION HOLES | 2.99      |
+------------------+-----------+
```

---

82. COALESCE / IFNULL

Handle NULLs. IFNULL(val, replacement) replaces NULL. COALESCE(val1, val2, val3) returns first non NULL. COALESCE is more flexible.

Query:

```
SELECT address, IFNULL(address2, 'N/A') AS line2 FROM address LIMIT 3;
```

Output:

```
+-------------------------+------+
| address                 | line2|
+-------------------------+------+
| 47 MySakila Drive       | N/A  |
| 28 MySQL Boulevard      | N/A  |
| 23 Workhaven Lane       | N/A  |
+-------------------------+------+
```

---

83. Conditional Logic with CASE / IF

CASE handles multiple conditions. IF(condition, true_val, false_val) is a shortcut for two way checks. IIF is SQL Server only.

Query:

```
SELECT title, IF(length > 120, 'Long', 'Short') AS duration FROM film LIMIT 3;
```

Output:

```
+------------------+----------+
| title            | duration |
+------------------+----------+
| ACADEMY DINOSAUR | Short    |
| ACE GOLDFINGER   | Short    |
| ADAPTATION HOLES | Short    |
+------------------+----------+
```

---

84. Regular Expressions in SQL

REGEXP does pattern matching more powerful than LIKE. ^ start, $ end, | or, [abc] character set. MySQL specific.

Query:

```
SELECT first_name FROM actor WHERE first_name REGEXP '^A|^B' LIMIT 5;
```

Output:

```
+------------+
| first_name |
+------------+
```

```
| BOB        |
| BURT       |
| ADAM        |
| AL         |
| ALAN       |
+------------+
```

---

85. User Defined Functions (UDFs)

Custom functions created with CREATE FUNCTION. Takes inputs, returns one value. Reusable. Use DETERMINISTIC if same input always gives same output.

Query:

```
DELIMITER //
CREATE FUNCTION rental_label(rate DECIMAL(4,2))
RETURNS VARCHAR(10) DETERMINISTIC
BEGIN
    IF rate <= 0.99 THEN RETURN 'Budget';
    ELSEIF rate <= 2.99 THEN RETURN 'Standard';
    ELSE RETURN 'Premium';
    END IF;
END //
DELIMITER ;
-- SELECT title, rental_label(rental_rate) FROM film LIMIT 3;
```

Output:

```
+------------------+----------------+
| title            | rental_label   |
+------------------+----------------+
| ACADEMY DINOSAUR | Budget         |
| ACE GOLDFINGER   | Premium        |
| ADAPTATION HOLES | Standard       |
+------------------+----------------+
```